Syntax and Design

CS-3180

Where are we at?

- Languages can either be compiled or interpreted
- Or something in between
- What other considerations go into a language?

Design

What actually differentiates these two pieces of code?

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

```python
print("Hello, World!")
```

Design

- The rules of each language are different
- Arrange the words differently
- Structure + combination of symbols -> SYNTAX

Keywords

- Languages often have special, reserved words
- Directives to the language as it runs
- Let's pick some apart in a Java program
- "Syntax Highlighting" makes this easier

Keywords

- On a word-by-word basis
- Some of these are more fundamental
- Some are words that *we* create
- Let's look at a list of Java keywords

```
import keyword
print(keyword.kwlist)
```

```java
public class Example {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        System.out.print("Enter your birth year: ");
        int birthYear = scanner.nextInt();

        int currentYear = 2026;
        int age = currentYear - birthYear;

        System.out.println("You are approximately " + age);

        // comment example
        if (age >= 18) {
            System.out.println("Status: You are an adult.");
        } else {
            System.out.println("Status: You are a child.");
        }
    }
}
```

Standard Library

- Things that "feel" reserved but are not
- E.g. extremely common functions
- Are often implemented in a library that ships with the language itself
- Often written IN the target language (E.g. String)
- Let's look at that example again

```java
public class Example {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        System.out.print("Enter your birth year: ");
        int birthYear = scanner.nextInt();

        int currentYear = 2026;
        int age = currentYear - birthYear;

        System.out.println("You are approximately " + age);

        // comment example
        if (age >= 18) {
            System.out.println("Status: You are an adult.");
        } else {
            System.out.println("Status: You are a child.");
        }
    }
}
```

Design

- Languages have similar concepts
- Implemented with different syntax
- To learn more Python, let's compare a few of them
- From a perspective of syntax comparison

Python Conditionals

- Whitespace sensitive
- `if`, `elif`, `else`: keywords
- Colon after statement
- No parentheses
- Thoughts?

```
if color == "red":
    print("Stop")
elif color == "yellow":
    print("Slow down")
elif color == "green":
    print("Go")
else:
    print("Invalid signal")
```

Java Conditionals

```
if ("red".equals(color)) {
    System.out.println("Stop");
} else if ("yellow".equals(color)) {
    System.out.println("Slow down");
} else if ("green".equals(color)) {
    System.out.println("Go");
} else {
    System.out.println("Invalid signal");
}
```

Python Loops

```
for i in range(5):
    print(i)

names = ["Alice", "Bob", "Charlie"]
for name in names:
    print(name)

while True:
    print("hi!")
```

Java Loops

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
for (String name : names) {
    System.out.println(name);
}

while (true) {
    System.out.println("hi!");
}
```

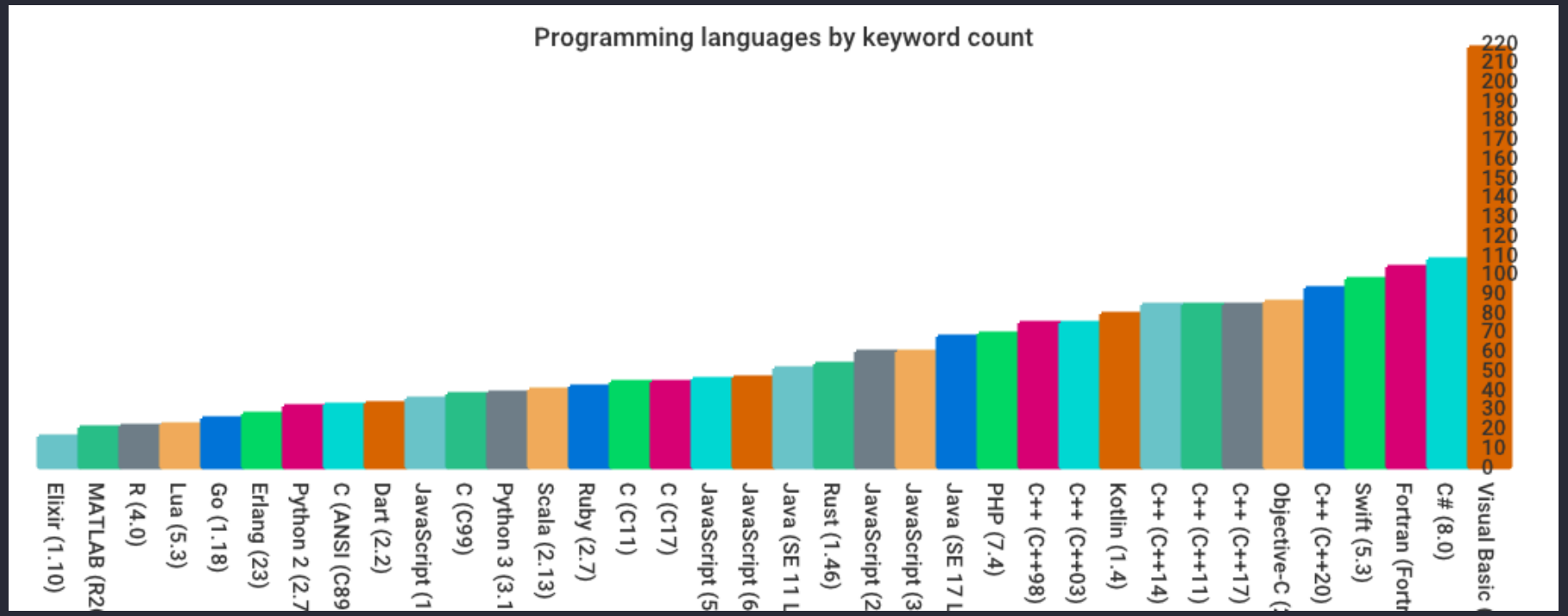
Python Lists

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])   # apple
print(fruits[-1])  # cherry -> last item

numbers = [1, 2]
numbers.append(3) # -> [1, 2, 3]
numbers.insert(1, 99) # 'insert_at_index' -> [1, 99, 2, 3]
```

Interesting Question

- Do you think it is possible for a language to have NO keywords at all?
- What would that even look like?
- It would have to be almost 100% std. library
- Lisp accomplishes this



https://github.com/e3b0c442/keywords

Paradigms Design

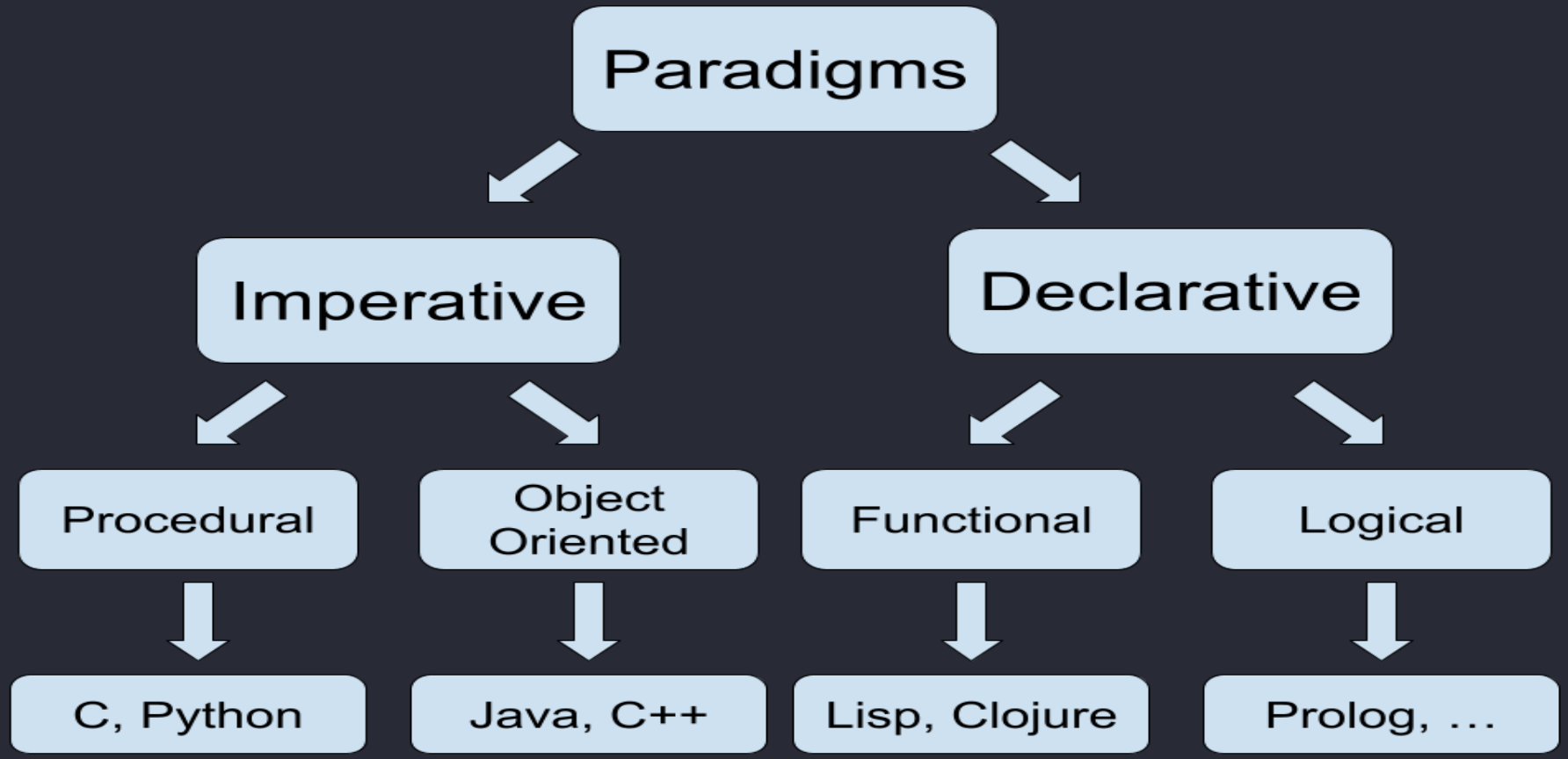
- Total keyword count says something about the language
- Bigger Picture =>
- Paradigm: High level way of conceptualizing the organization and execution of your program
- Almost every modern language is "multi-paradigm"

Paradigms

- Generally two main groups
- Imperative => (Procedural, Object Oriented)
- Step-by-step instructions to follow

- Declarative => (Functional, Logical)
- Describe the result you want

- Skipping details for now, I will call this out as it arises



Skipping over many details here

Python -> Generally Procedural

Java -> Generally Object-Oriented

Other Considerations

- Language Ecosystem
- Build System?
- Dependency Management
- Concurrency Model

Type System!

- Elephant in the room
- Languages take a different approach to variable types
- Some are much more loose with the rules than others
- Next lecture, we will dive head first into this

Questions?